

Planejamento Lógico TaskTracker

Etapa Inicial — Bootcamp II

Identificação	Informação
Aluno	Eltern Camargos Vieira
Curso	Ciência de Dados e Machine Learning
RA	22553715
E-mail institucional	eltern.camargos@sempreceub.com

1. Apresentação e entendimento do problema

1.1 Identificação do projeto

O projeto TaskTracker consiste no planejamento de um sistema simples para gerenciamento de tarefas. O sistema será desenvolvido nas próximas etapas do Bootcamp II e terá como objetivo ajudar o usuário a registrar, consultar e acompanhar suas atividades de maneira organizada.

1.2 Problema a ser resolvido

No dia a dia, uma pessoa pode precisar lembrar compromissos acadêmicos, profissionais e pessoais. Quando essas informações ficam apenas na memória, em mensagens ou em anotações espalhadas, aumenta o risco de esquecer prazos, repetir tarefas ou perder a noção do que já foi concluído. Em uma equipe, a falta de organização também dificulta saber quais atividades ainda estão pendentes e quais devem receber atenção primeiro.

O problema central é, portanto, a ausência de um local único e simples para registrar as tarefas, estabelecer sua prioridade, informar um prazo e acompanhar seu andamento.

1.3 Solução proposta

A solução proposta é o TaskTracker, um programa que permitirá cadastrar, visualizar, editar, concluir e excluir tarefas. Cada tarefa terá título, descrição, prioridade, prazo e situação. Dessa forma, o usuário poderá consultar suas atividades, identificar as mais urgentes e acompanhar o que ainda precisa ser feito.

Na primeira versão em Python, o sistema poderá funcionar em um menu de texto. O usuário escolherá uma opção, informará os dados solicitados e receberá uma mensagem confirmando

o resultado da operação. A estrutura foi planejada para ser simples no início e permitir melhorias futuras.

1.4 Objetivo geral

Planejar e, nas próximas fases, desenvolver um sistema em Python capaz de organizar tarefas e apresentar ao usuário informações claras sobre prazos, prioridades e andamento das atividades.

1.5 Objetivos específicos

- Cadastrar tarefas com informações essenciais.
- Exibir todas as tarefas cadastradas de forma organizada.
- Permitir a busca e a alteração de uma tarefa.
- Marcar uma tarefa como concluída.
- Excluir uma tarefa quando ela não for mais necessária.
- Apresentar resumos, como quantidade de tarefas pendentes e concluídas.
- Organizar os arquivos do projeto em um repositório GitHub.

2. Decomposição do problema e dados do sistema

2.1 Funcionalidades principais

Funcionalidade	Finalidade
Cadastrar tarefa	Registrar uma nova atividade e seus dados.
Listar tarefas	Mostrar todas as tarefas cadastradas.
Consultar tarefa	Localizar uma tarefa pelo seu identificador ou título.
Editar tarefa	Alterar informações cadastradas.
Concluir tarefa	Mudar a situação de pendente para concluída.
Excluir tarefa	Remover uma tarefa após confirmação.
Exibir resumo	Informar totais de tarefas pendentes e concluídas.

2.2 Entradas de dados

As entradas são as informações digitadas pelo usuário para que o sistema realize uma operação.

Entrada	Tipo previsto	Exemplo
---------	---------------	---------

Título	Texto obrigatório	Entregar atividade do Bootcamp
Descrição	Texto opcional	Revisar o documento e enviar o PDF
Prioridade	Opção: Alta, Média ou Baixa	Alta
Data de prazo	Data no formato DD/MM/AAAA	30/08/2026
Situação	Pendente ou Concluída	Pendente
Identificador	Número inteiro gerado pelo sistema	1
Opção do menu	Número inteiro	2 — Listar tarefas

2.3 Saídas de dados

- Menu com as funcionalidades disponíveis.
- Confirmação de tarefa cadastrada, editada, concluída ou excluída.
- Lista organizada com identificador, título, prioridade, prazo e situação.
- Aviso quando um dado obrigatório estiver vazio ou inválido.
- Aviso quando uma tarefa não for encontrada.
- Quantidade total de tarefas, tarefas pendentes e tarefas concluídas.

2.4 Regras do sistema

- O título da tarefa é obrigatório e não pode conter apenas espaços.
- Cada tarefa deve possuir um identificador único, criado automaticamente pelo sistema.
- A prioridade deve ser somente Alta, Média ou Baixa.
- A data deve seguir o formato DD/MM/AAAA e representar uma data válida.
- Toda nova tarefa deve iniciar com a situação Pendente.
- Uma tarefa só poderá ser editada, concluída ou excluída se existir.
- Antes da exclusão, o sistema deverá solicitar confirmação do usuário.
- A situação poderá ser apenas Pendente ou Concluída.
- Quando não houver tarefas cadastradas, o sistema deverá informar isso claramente.
- O menu continuará sendo exibido até o usuário escolher a opção Sair.

3. Receita lógica do sistema — algoritmo em português

A seguir está o funcionamento planejado do TaskTracker em linguagem simples. Esse algoritmo servirá como base para a implementação em Python.

3.1 Fluxo principal

1. Iniciar o programa.
2. Criar uma lista vazia para guardar as tarefas.
3. Exibir o menu com as opções: cadastrar, listar, consultar, editar, concluir, excluir, exibir resumo e sair.
4. Pedir ao usuário que escolha uma opção.
5. Verificar se a opção digitada é válida.
6. Executar a funcionalidade escolhida.
7. Ao terminar a operação, voltar ao menu principal.
8. Repetir o processo até que o usuário escolha Sair.
9. Exibir uma mensagem de encerramento e finalizar o programa.

3.2 Cadastro de tarefa

10. Pedir o título da tarefa.
11. Retirar espaços desnecessários do início e do final do título.
12. Se o título estiver vazio, avisar o usuário e pedir novamente.
13. Pedir uma descrição; se o usuário não informar, aceitar o campo vazio.
14. Pedir a prioridade: Alta, Média ou Baixa.
15. Se a prioridade for inválida, mostrar as opções permitidas e pedir novamente.
16. Pedir a data de prazo no formato DD/MM/AAAA.
17. Verificar se a data informada é válida.
18. Gerar automaticamente um identificador numérico.
19. Definir a situação inicial como Pendente.
20. Guardar a nova tarefa na lista de tarefas.
21. Mostrar a mensagem “Tarefa cadastrada com sucesso”.

3.3 Visualização das tarefas

22. Verificar se a lista de tarefas está vazia.
23. Se estiver vazia, mostrar “Nenhuma tarefa cadastrada”.
24. Caso existam tarefas, ordenar primeiro pela situação, depois pela prioridade e pelo prazo.
25. Percorrer a lista e apresentar cada tarefa com identificador, título, descrição, prioridade, prazo e situação.
26. Ao finalizar a exibição, voltar ao menu principal.

3.4 Consulta de uma tarefa

27. Pedir o identificador da tarefa.
28. Verificar se o valor digitado é um número inteiro.
29. Procurar na lista uma tarefa com esse identificador.
30. Se encontrar, mostrar todos os dados da tarefa.

31. Se não encontrar, informar que a tarefa não existe.

3.5 Edição de tarefa

- 32. Pedir o identificador da tarefa que será editada.
- 33. Procurar a tarefa na lista.
- 34. Se ela não existir, informar o usuário e encerrar a operação.
- 35. Mostrar os dados atuais.
- 36. Pedir os novos dados, permitindo manter um valor ao pressionar Enter.
- 37. Validar novamente título, prioridade e data, quando forem alterados.
- 38. Salvar as alterações.
- 39. Mostrar a mensagem “Tarefa atualizada com sucesso”.

3.6 Conclusão de tarefa

- 40. Pedir o identificador da tarefa.
- 41. Localizar a tarefa correspondente.
- 42. Se ela não existir, mostrar uma mensagem de erro.
- 43. Se já estiver concluída, informar que nenhuma alteração é necessária.
- 44. Caso esteja pendente, alterar a situação para Concluída.
- 45. Confirmar que a tarefa foi concluída.

3.7 Exclusão de tarefa

- 46. Pedir o identificador da tarefa.
- 47. Localizar a tarefa na lista.
- 48. Se não encontrar, informar o usuário.
- 49. Se encontrar, mostrar o título e pedir confirmação: Sim ou Não.
- 50. Se a resposta for Sim, remover a tarefa e confirmar a exclusão.
- 51. Se a resposta for Não, cancelar a operação sem alterar os dados.

3.8 Resumo das tarefas

- 52. Contar quantas tarefas existem na lista.
- 53. Contar quantas estão pendentes.
- 54. Contar quantas estão concluídas.
- 55. Contar quantas possuem prioridade Alta.
- 56. Mostrar os resultados de forma organizada.

3.9 Tratamento de erros

Sempre que o usuário informar um valor inválido, o sistema deverá explicar o problema e permitir uma nova tentativa, evitando que o programa seja encerrado inesperadamente.

Exemplos: texto no lugar de uma opção numérica, prioridade diferente das opções disponíveis ou data inexistente.